

Informe — Trabajos Prácticos 1 y 2

Curso: Protocolos de Comunicación — Fundación Fulgor

Docentes: Facundo Sposetti, Jorge Manuel Finochietto

Alumno: Francisco Javier Vásquez

Fecha: Agosto de 2026

Índice

1. Introducción
 2. TP1 — Proyecto LEDs
 3. TP2 — Generador LFSR
 4. Conclusiones generales
-

Introducción

Este informe documenta la resolución de los Trabajos Prácticos 1 y 2 del curso de Protocolos de Comunicación. Ambos trabajos abordan el diseño, la implementación y la verificación de módulos digitales en Verilog:

- **TP1** aborda el flujo de trabajo de diseño digital sobre FPGA: un sistema simple (contador + registro de desplazamiento que controla un patrón de LEDs), su verificación funcional mediante testbench, y su síntesis, implementación y generación de bitstream en **Xilinx Vivado**, con los IP cores de depuración **VIO** (Virtual Input/Output) e **ILA** (Integrated Logic Analyzer) ya instanciados en el código. La programación final sobre la FPGA física (vía el servidor remoto de la fundacion) quedó pendiente por un problema de acceso a ese servidor. El diseño RTL fue resuelto en términos generales de forma conjunta entre todos los cursantes, guiados por el docente; el trabajo propio se concentró en el diseño de la batería de tests del testbench.
- **TP2** profundiza en el diseño de un **generador de secuencias pseudoaleatorias (LFSR)** y de un módulo verificador de esa secuencia (**checker**), con énfasis en el diseño de máquinas de estado y el manejo de señales de control (`valid`, resets sincrónico/asincrónico). Para este TP, se abordó tanto el diseño (generador, checker) como la verificación (testbenches y baterías de tests) de manera más individual, pero guiada por el docente.

Este informe reconstruye y explica el razonamiento de diseño, las decisiones tomadas y los resultados obtenidos a partir del código y los testbenches finales.

TP1 — Proyecto LEDs

1. Objetivo

El objetivo del trabajo es:

- Diseñar un sistema (`top`) que controle un patrón de LEDs mediante un contador y un registro de desplazamiento (shift register).
- El sistema debe incluir instancias de los IP cores **VIO** e **ILA** de Xilinx para control y debug del módulo top desde la herramienta Vivado, una vez implementado en la FPGA.
- Especificaciones funcionales:
 - `i_reset` : reset del sistema que pone en cero al contador e inicializa el shift register.
 - `i_sw[0]` : habilita (1) o detiene (0) el conteo, sin alterar el estado actual del contador ni del shift register.
 - El shift register se desplaza únicamente cuando el contador alcanza uno de cuatro límites configurables (`R0` – `R3`), seleccionados en cualquier momento mediante `i_sw[2:1]` .
 - `i_sw[3]` selecciona el color de los LEDs RGB (verde o azul).

2. Arquitectura del diseño

El sistema se estructuró en tres módulos, integrados por un módulo top (`top_leds`):

```
top_leds
├─ counter      (contador con 4 límites configurables)
├─ shift_reg    (registro de desplazamiento circular de 4 bits)
├─ vio_0        (Xilinx VIO – solo en síntesis)
└─ ila_0        (Xilinx ILA – solo en síntesis)
```

counter.v — Contador de 32 bits con cuatro límites (`limit_0` .. `limit_3`), seleccionados por `i_sel_count_limit` (2 bits). Al llegar al límite seleccionado, vuelve a 0 y genera un pulso de un ciclo en `o_shift` :

```
assign counter_next = ((i_sel_count_limit == 2'b00) && (counter >= limit_0)) ? 'd0 :
                      ((i_sel_count_limit == 2'b01) && (counter >= limit_1)) ? 'd0 :
                      ((i_sel_count_limit == 2'b10) && (counter >= limit_2)) ? 'd0 :
                      ((i_sel_count_limit == 2'b11) && (counter >= limit_3)) ? 'd0 :
                                                                counter + 1'b1;

...
assign o_shift = (counter == 'd0);
```

El reset (`i_reset` , activo bajo internamente — `negedge i_reset` en la sensitivity list) es **asíncrono** y lleva el contador a 0. La habilitación (`i_enable`) congela el contador en su valor actual cuando está en 0, sin resetearlo — esto es clave para que el TEST1 del testbench pase (verifica que deshabilitar el conteo no reinicia el estado).

Un detalle de diseño relevante: `o_shift` se define como `counter == 0` , de modo que **el primer pulso de shift ocurre inmediatamente después del reset** (cuando el contador arranca en 0), y luego cada `limit + 1` ciclos. Este comportamiento se verifica explícitamente en TEST3.

shift_reg.v — Registro de desplazamiento circular de 4 bits que rota un único bit encendido (`4'b1000` tras el reset). Solo rota cuando `i_shift` (proveniente de `o_shift` del contador) e `i_enable` están

ambos activos:

```
always@(posedge clock or negedge i_reset)
begin
    if      (!i_reset)
        shift_register <= 4'b1000;
    else if (i_shift && i_enable)
        shift_register <= {shift_register[0], shift_register[NB_SHIFT_REG-1:1]};
end
```

top_leds.v — Integra ambos módulos y agrega la lógica de colorización RGB:

```
assign o_led    = led_enable;
assign o_led_b = (sw3_int) ? led_enable : 4'b0000;
assign o_led_g = (!sw3_int) ? led_enable : 4'b0000;
```

Es decir, `o_led` siempre refleja el patrón del shift register, y ese mismo patrón se enruta hacia el canal azul o verde según `i_sw[3]`, quedando el canal no seleccionado en 0.

Instancias VIO/ILA: para permitir el control y la observación del sistema una vez implementado en la FPGA (sin recompilar el bitstream cada vez que se quiere cambiar un estímulo), `top_leds` incorpora un mux `sel_mux` que decide si las entradas de control provienen de los switches físicos (`i_sw`, `i_reset`) o de los registros virtuales expuestos por el VIO (`vio_sw`, `vio_sw3`, `vio_reset`):

```
assign sw_int    = sel_mux ? vio_sw    : i_sw[2:0];
assign sw3_int   = sel_mux ? vio_sw3   : i_sw[3] ;
assign reset_int = sel_mux ? vio_reset : i_reset ;
```

El VIO expone como entradas (monitoreadas en el dashboard de Vivado) los tres buses de LEDs (`o_led`, `o_led_b`, `o_led_g`), y como salidas los controles virtuales (`sel_mux`, `vio_reset`, `vio_sw[2:0]`, `vio_sw3`). El ILA, por su parte, captura los mismos tres buses de LEDs para poder observar su evolución temporal directamente en hardware.

El VIO y el ILA son cores de debug pensados para ser manejados desde el dashboard de Vivado vía JTAG una vez que la FPGA ya está programada: sus entradas de control (`probe_out0..3`) las mueve un usuario interactuando con el hardware, no un testbench. Por eso, instanciarlos en una simulación no aporta nada — quedarían estaticos en su valor inicial sin importar qué simulador se use, incluido el propio de Vivado. Para que el mismo archivo sirva tanto para simulación como para síntesis, se usó una directiva de compilación condicional:

```

`ifdef SYNTHESIS
    // instancias de vio_0 e ila_0
`else
    // en simulación, VIO "bypasado": Las entradas físicas pasan directo
    assign sel_mux    = 1'b0;
    assign vio_reset = 1'b1;
    ...
`endif

```

3. Verificación (testbench)

Esta es la parte del TP1 que correspondió más directamente a este trabajo: a partir del RTL ya definido en clase, se diseñó el testbench y la batería de tests que lo ejercitan.

El testbench (`top_leds_tb.v`) instancia `top_leds` y provee:

- Generación de clock (`always #5 clock = ~clock`).
- Una `task reset()` que asierta el reset de inmediato, lo mantiene por un tiempo aleatorio (entre 1 y 100 unidades de tiempo) y luego lo desasierta sincronizado con el clock — emulando un reset de duración variable, similar al pedido de la consigna del TP2 pero aplicado aquí al reset del sistema de LEDs.
- Un mecanismo de selección de test vía ``define TESTn` que incluye ( ``include` ) el archivo correspondiente de la carpeta `tests/`, permitiendo compilar y correr cada test de forma independiente.

Se implementaron **6 tests**, cada uno enfocado en un aspecto funcional distinto del sistema:

Test	Qué verifica
TEST1	Con <code>i_sw[0] = 0</code> (conteo deshabilitado), <code>o_led</code> no cambia durante un número aleatorio de ciclos (50–500), confirmando que deshabilitar no altera el estado.
TEST2	El sistema es puramente sincrónico: con el clock forzado en 0 (congelado), ninguna señal de estado (<code>o_led</code> , <code>o_led_b</code> , <code>o_led_g</code> , <code>counter</code> , <code>shift_register</code>) cambia.
TEST3	El período entre shifts coincide exactamente con el límite configurado (<code>limit + 1</code> ciclos), medido para los 4 valores de <code>i_sw[2:1]</code> y verificado durante 5 períodos consecutivos por cada uno.
TEST4	Comportamiento al cambiar el límite en caliente: Caso A (límite mayor→menor, con <code>counter</code> ya por encima del nuevo límite) el contador debe resetearse y el shift register debe rotar; Caso B (límite menor→mayor) el contador debe seguir incrementando normalmente, sin reset ni shift extra.
TEST5	<code>i_sw[3]</code> controla el canal de color: con <code>sw[3]=1</code> , <code>o_led_b == o_led</code> y <code>o_led_g == 0</code> ; con <code>sw[3]=0</code> , es al revés. Además confirma que <code>o_led</code> en sí no depende de <code>sw[3]</code> .
TEST6	Reset asíncrono: con el clock congelado, al asertar <code>i_reset</code> el contador y el shift register vuelven a su estado inicial sin necesidad de un flanco de clock , y <code>o_led</code> lo refleja inmediatamente.

Todos los tests usan `force` / `release` sobre señales internas (por ejemplo, forzar los límites del contador a valores pequeños como 16, 32, 64 y 128 ciclos, en lugar de los valores reales de milisegundos) para poder verificar el comportamiento en tiempos de simulación razonables.

Cada test finaliza con un mensaje `$display` indicando `PASS` o `FAIL` , lo que permite verificar rápidamente el resultado de cada corrida en la consola del simulador.

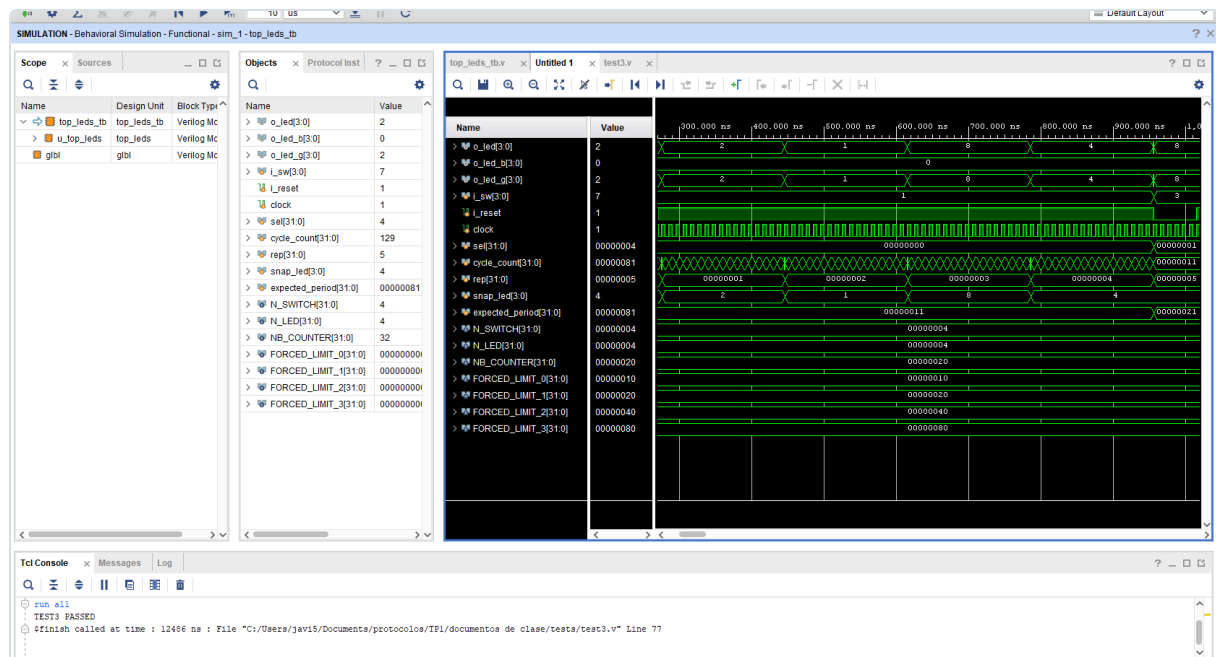


Figura 1. Simulación de comportamiento (Vivado, TEST3) mostrando `o_led`/`o_led_b`/`o_led_g` transicionando junto con `sel` (límite seleccionado) y `expected_period` (0x11→0x21, es decir 17→33 ciclos al pasar de `limit_0` a `limit_1`). La consola confirma `TEST3 PASSED`.

4. Síntesis, implementación y bitstream

El flujo seguido en Vivado (documentado en las diapositivas del curso) fue:

1. Creación de un nuevo proyecto Vivado, seleccionando la placa objetivo (Arty A7-35, según el archivo de *constraints* `Arty-A7-35-Master.xdc` incluido en el repositorio).
2. Incorporación de las fuentes de diseño (`counter.v`, `shift_reg.v`, `top_leds.v`), el archivo de *constraints* (`.xdc`) y las fuentes de simulación (`top_leds_tb.v` + `tests/*.v`).
3. Configuración e instanciación de los IP cores VIO e ILA desde el IP Catalog, con los parámetros indicados en los comentarios de `top_leds.v` (3 probes de entrada de 4 bits para el VIO, 3 probes de 4 bits para el ILA con profundidad de muestreo 1024).
4. Síntesis, implementación y generación del bitstream — completado sin errores en Vivado local.

5. Conclusiones del TP1

El trabajo permitió recorrer gran parte del flujo de diseño digital: desde la descripción RTL de un sistema con control de tiempo (contador con límites configurables) y control de estado visual (shift register), pasando por una verificación funcional rigurosa mediante testbench con manipulación directa de señales internas, hasta la síntesis, implementación y generación de bitstream en Vivado con los IP cores de VIO/ILA ya instanciados. La programación real sobre la FPGA y la interacción con el VIO/ILA en hardware quedaron pendientes por el problema de acceso al servidor remoto mencionado en la sección anterior. La separación entre lógica de simulación y de síntesis resultó útil para poder verificar el diseño mediante testbench sin depender de los IP cores propietarios, ya que el VIO y el ILA no tienen manera de ser manejados por un testbench.

TP2 — Generador LFSR

1. Objetivo

Este trabajo pide diseñar y verificar un generador de secuencias pseudoaleatorias basado en un **LFSR (Linear Feedback Shift Register)**, junto con un módulo que verifique la validez de esa secuencia. Las actividades pedidas son:

- **Actividad 1:** usar el software provisto (`LFSRTestBench.exe`) para encontrar la configuración de polinomio que genere la secuencia de mayor período posible, e implementar el generador en Verilog con seed configurable, señal de habilitación `i_valid`, reset asíncrono `i_rst` (carga un seed fijo) y reset síncrono `i_soft_reset` (carga `i_seed`).
- **Actividad 2:** construir el testbench con clock de 10 MHz, reset de duración aleatoria (entre 1 μ s y 250 μ s), señal de valid aleatoria ciclo a ciclo, y tasks para cambiar la seed y para pulsar cada uno de los dos resets por un tiempo aleatorio.
- **Actividad 3:** verificar la periodicidad del generador, tanto con la seed por defecto como con seeds aleatorias.
- **Actividad 4:** implementar un **LFSR Checker** que se conecte a la salida del generador y determine, ciclo a ciclo, si la secuencia recibida es válida, con una señal `o_lock` que se activa tras 5 aciertos consecutivos y se desactiva tras 3 errores consecutivos.
- **Actividad 5:** integrar el checker al testbench, agregar un monitor de cambios de `o_lock`, y verificar tráfico válido continuo, las condiciones de frontera de lock/unlock, transiciones repetidas, y recuperación tras reset aleatorio del checker.

2. Marco teórico: LFSR Fibonacci vs. Galois

La consigna presenta las dos topologías clásicas de LFSR:

- **Fibonacci LFSR:** la realimentación se calcula con una cadena de XOR entre varios bits de salida (los “taps”), y el resultado se inyecta únicamente en la primera posición del registro.
- **Galois LFSR:** en lugar de una única cadena de XOR externa, se insertan compuertas XOR *internas* en cada posición de tap, todas alimentadas por el mismo bit de realimentación (el bit más significativo saliente). Esta topología es la elegida para la implementación.

Se utilizó el ejecutable provisto por el curso (`LFSRTestbench/LFSRTestbench.exe`) para determinar *un* polinomio que maximice el período de la secuencia. Para un registro de 16 bits, el período máximo posible (secuencia de longitud completa o “maximal length sequence”) es $2^{16} - 1 = \mathbf{65535}$, y se alcanza con un polinomio primitivo. El polinomio elegido fue:

$$x^{16} + x^{14} + x^{13} + x^{11} + 1$$

correspondiente a taps en los bits 11, 13 y 14 (más el bit 15, MSB, como fuente de realimentación). Este polinomio es efectivamente primitivo: el TEST1 del testbench del generador mide el período real y confirma que es exactamente 65535.

Como referencia de partida (código provisto por el ejecutable) se usó `primer_archivo.v`: una implementación mínima del LFSR Galois de 16 bits, sin resets ni control de valid, que sirvió como

referencia para confirmar que la topología (ubicación de los taps) estaba bien implementada antes de construir la versión parametrizada y controlable.

3. Diseño e implementación

`rtl/lfsr_generator.v` — Módulo RTL sintetizable con todas las señales de control pedidas:

Puerto	Dirección	Descripción
<code>i_clk</code>	in	Clock del sistema
<code>i_rst</code>	in	Reset asíncrono — carga <code>DEFAULT_SEED</code>
<code>i_soft_reset</code>	in	Reset síncronico — carga <code>i_seed</code>
<code>i_valid</code>	in	El LFSR solo avanza cuando está en alto
<code>i_seed</code>	in	Seed configurable en tiempo de ejecución
<code>o_data</code>	out	Estado actual del registro
<code>o_valid</code>	out	Compañera registrada de <code>o_data</code>

La lógica de próximo estado (`lfsr_next`) es puramente combinacional, separada del bloque secuencial, y la parte secuencial resuelve la prioridad de control `i_rst > i_soft_reset > i_valid > hold`:

```
always @(posedge i_clk or posedge i_rst) begin
    if (i_rst) begin
        lfsr    <= DEFAULT_SEED;
        o_valid <= 1'b0;
    end else if (i_soft_reset) begin
        lfsr    <= i_seed;           // Load runtime seed synchronously
        o_valid <= 1'b0;           // reseed, not a PRBS advance
    end else if (i_valid) begin
        lfsr    <= lfsr_next;       // advance sequence
        o_valid <= 1'b1;
    end else begin
        o_valid <= 1'b0;           // hold — no valid token, no reset
    end
end
end
```

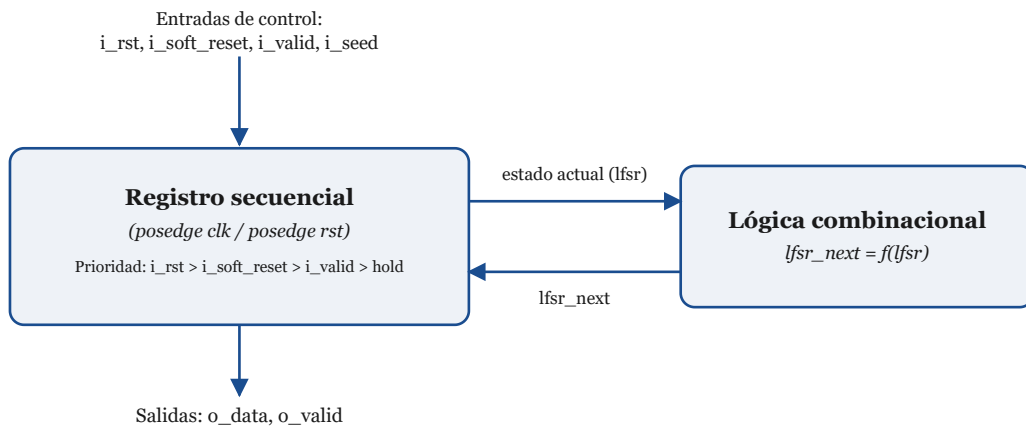



Figura 2. El registro secuencial concentra la prioridad de control ($i_rst > i_soft_reset > i_valid > hold$); la lógica combinacional solo calcula el próximo valor del LFSR a partir del estado actual, sin ver las señales de control.

Corolario — el dato y el valid siempre viajan juntos: la consigna no pide explícitamente una señal de valid de salida, pero se agregó porque el par dato/valid debe viajar siempre juntos. Este es un punto sobre el que se hizo bastante énfasis en el curso: si el dato se registra (se atrasa) N veces a lo largo de un camino, el valid que lo acompaña debe registrarse exactamente esas mismas N veces. Cualquier asimetría entre ambos caminos introduce un desfase entre el dato y su valid — el consumidor terminaría calificando como válido un dato viejo, o como inválido un dato que en realidad sí lo es.

En este módulo, `o_valid` se implementa como flip-flop que sigue exactamente la misma cadena de prioridad que `lfsr`, para que dato y valid avancen (o se congelen) juntos ciclo a ciclo: queda en 0 tras cualquiera de los dos resets (el dato resultante es una recarga, no un avance de secuencia) y en 1 únicamente cuando la rama de `i_valid` ganó ese ciclo — incluso si `i_valid` está en alto simultáneamente con `i_soft_reset`, ya que este último tiene prioridad. Esto se verifica puntualmente en el TEST oc del testbench.

Esta misma disciplina es, en general, la que permite que una señal de valid funcione como mecanismo de sincronización entre bloques que corren a clocks distintos (por ejemplo, en un cruce de dominios de reloj o en una interfaz entre un generador y un receptor más lento): el emisor mantiene el dato y su valid estables durante una ventana proporcional a la diferencia entre ambos períodos de clock, de forma que el dominio más lento tenga garantizado al menos un flanco propio para muestrear ese dato mientras el valid sigue arriba. Si el valid no acompañara fielmente al dato en esa ventana, el receptor podría muestrear un dato ya reemplazado (perdiendo la muestra) o interpretar como válido un dato que ya no lo es, rompiendo la comunicación entre ambos bloques.

`rtl/lfsr_checker.v` — Módulo verificador con una máquina de estados de **dos** estados (UNLOCKED / LOCKED):

```

localparam ST_UNLOCKED = 1'b0;
localparam ST_LOCKED   = 1'b1;

```

- **UNLOCKED:** compara `i_data` contra el valor predicho por una copia interna del LFSR (`lfsr_ref`, avanzada con la misma función `lfsr_step` y el mismo polinomio que el generador). Si

acierta, acumula `valid_cnt`; al llegar a 5 pasa a `LOCKED`. En caso de fallar, se toma el dato de ese ciclo como nueva referencia para el próximo chequeo.

- **LOCKED:** acumula errores consecutivos (`invalid_cnt`); al llegar a 3 vuelve a `UNLOCKED` y fuerza la referencia interna al “centinela”. Si falla algún chequeo, se continúa calculando la referencia a partir del valor predicho (no se reseedea con el dato recibido en `i_data`).

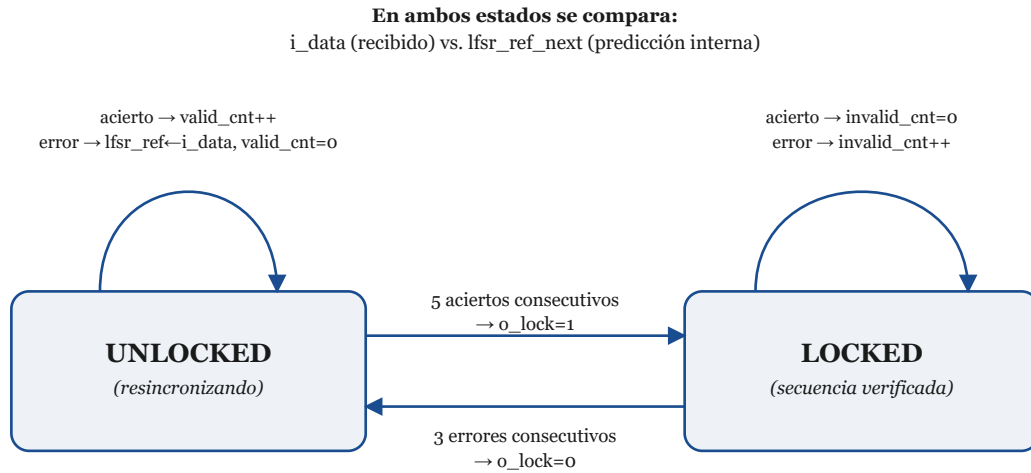


Figura 3. Máquina de estados de dos estados del LFSR Checker: pasa a `LOCKED` tras 5 aciertos consecutivos y vuelve a `UNLOCKED` tras 3 errores consecutivos.

Decisión de diseño — el “centinela” en `lfsr_ref`: la referencia interna se inicializa en `0` (un valor que la secuencia PRBS real nunca produce, ya que no pertenece al ciclo maximal de $2^{16}-1$ estados no nulos), tanto en el reset como al desbloquear, en lugar de sembrarla con `DEFAULT_SEED`. Esto garantiza que la primera comparación tras cualquiera de esos eventos falle **de manera intencional**, disparando la resincronización normal, en vez de “arriesgarse” a que el valor seedeadado coincida por casualidad con el dato real recibido — lo que haría lockear al checker un ciclo antes de lo esperado por pura casualidad.

```

if (i_rst) begin
    state      <= ST_UNLOCKED;
    lfsr_ref    <= {DATA_WIDTH{1'b0}}; // forces the first comparison to fail
    valid_cnt   <= 4'd0;
    invalid_cnt <= 4'd0;
    o_lock      <= 1'b0;
end else if (i_valid) begin
    if (state == ST_UNLOCKED) begin
        if (i_data == lfsr_ref_next) begin
            lfsr_ref <= lfsr_ref_next;

            if (valid_cnt == LOCK_THRESHOLD - 1) begin
                o_lock <= 1'b1;
                valid_cnt <= 4'd0;
                state <= ST_LOCKED;
            end else
                valid_cnt <= valid_cnt + 4'd1;
        end else begin
            lfsr_ref <= i_data;
            valid_cnt <= 4'd0;
        end
    end else begin // ST_LOCKED
        if (i_data == lfsr_ref_next) begin
            lfsr_ref <= lfsr_ref_next;
            invalid_cnt <= 4'd0;
        end else if (invalid_cnt == UNLOCK_THRESHOLD - 1) begin
            o_lock <= 1'b0;
            invalid_cnt <= 4'd0;
            state <= ST_UNLOCKED;
            lfsr_ref <= {DATA_WIDTH{1'b0}}; // sentinel, same as on reset
        end else begin
            lfsr_ref <= lfsr_ref_next; // keeps predicting despite the transient error
            invalid_cnt <= invalid_cnt + 4'd1;
        end
    end
end
end

```

4. Verificación

`tb/tb_lfsr_generator.v` (**Actividades 2 y 3**) — Testbench del generador aislado, con toda la infraestructura pedida:

- Clock de 10 MHz (período de 100 ns).
- `task_set_seed`: cambia `i_seed` en tiempo de simulación.
- `task_async_reset` / `task_soft_reset`: pulsar cada reset por una duración aleatoria entre 1 μ s y 250 μ s (`$urandom_range`).

- Proceso de valid aleatorio: `rand_valid` se randomiza en cada `posedge` cuando está habilitado; `i_valid` es un mux entre `forced_valid` (para tests determinísticos) y `rand_valid` (modo aleatorio), evitando que ambos intenten manejar la misma señal a la vez.

Tests implementados:

Test	Qué verifica
oa	<code>o_data == DEFAULT_SEED</code> justo después de <code>i_rst</code> , con <code>o_valid == 0</code> .
ob	<code>o_data == i_seed</code> justo después de <code>i_soft_reset</code> , con <code>o_valid == 0</code> .
oc	Prioridad: <code>i_soft_reset</code> e <code>i_valid</code> juntos → gana el reset y <code>o_valid</code> queda en 0.
1	Período del LFSR con <code>DEFAULT_SEED</code> : debe ser exactamente 65535.
2	Igual que el anterior, repetido con 5 seeds aleatorias distintas.
3	Con <code>i_valid = 0</code> durante 50 ciclos, la salida no avanza y <code>o_valid</code> permanece en 0.
4	Valid intermitente aleatorio durante 3000 ciclos, comparado ciclo a ciclo contra un modelo de referencia (<code>lfsr_step</code>) que solo avanza cuando el <code>i_valid</code> muestreado fue 1; también verifica que <code>o_valid</code> coincide con el <code>i_valid</code> muestreado un ciclo antes.

`tb/channel_delay.v` + `tb/tb_lfsr_system_channel.v` **(Actividad 5)** — Testbench del sistema completo: generador → canal → checker (`u_chk.i_valid` conectado a la salida del canal, no al `i_valid` del testbench, para que el par valid/dato viaje completo desde el generador hasta el checker). El canal (`channel_delay.v`) es un shift register de latencia configurable (`i_delay`, 0 a 31 ciclos) que modela distintas “distancias” de transmisión entre generador y checker: con `delay=0` es un passthrough combinacional puro, equivalente a conectar ambos módulos directo — esa es la instancia mínima que satisface la Actividad 5 — y con `delay>0` se extiende más allá de lo pedido por la consigna.

El canal recibe tres señales de control desde el testbench:

- `i_delay`: la latencia vigente. Cambiarla mientras hay datos en tránsito haría que la lectura “salte” a otra edad del buffer, así que el testbench siempre pulsa un reset del canal junto con el cambio de delay, para que arranque “vacío” en la nueva distancia (como reconectar el cable).
- `i_valid`: gateado aleatoriamente 50/50 en cada ciclo (mismo mecanismo `rand_valid` que en `tb_lfsr_generator.v`) en vez de mantenerse fijo en alto durante toda una ráfaga — las tasks `send_valid(n)` / `send_invalid(n)` tiran la moneda ciclo a ciclo hasta entregar exactamente `n` datos, así que el checker recibe siempre la misma cantidad de datos, solo que repartidos sobre una cantidad aleatoria de ciclos de reloj en vez de `n` ciclos consecutivos.
- `i_inject_error`: fuerza un dato incorrecto al checker sin alterar el generador.

Decisión de diseño — corrupción de un bit aleatorio, no de la palabra entera: este último punto se modificó deliberadamente a partir de una sugerencia recibida en un encuentro del curso, luego de mencionar que la implementación vigente en ese momento invertía la palabra completa (`~i_data`). La versión final invierte, en cambio, un **único bit elegido al azar** en cada ciclo de inyección: un solo bit volteado es mucho más representativo de un error real de transmisión (bit-flip por ruido en el canal) que invertir el dato entero de una vez. La propiedad que necesitan los tests de frontera —que con `i_inject_error=1` el checker vea siempre un dato distinto al esperado— se mantiene igual que antes: invertir un solo bit de un valor de `DATA_WIDTH` bits nunca puede reproducir ese mismo valor, sea cual sea el bit elegido.

Además, un monitor `always @(o_lock)` imprime un mensaje cada vez que cambia el estado de lock, indicando valor anterior y nuevo — tal como pide la consigna. Tasks auxiliares: `reset_generator` / `reset_checker` (reset aleatorio de cada módulo), `send_valid(n)` / `send_invalid(n)` (descriptas arriba) y `lock_checker` (lleva al checker a estado `LOCKED`).

Organización de los tests: a diferencia de `tb_lfsr_generator.v` (una única secuencia principal con todos los tests en cadena), `tb_lfsr_system_channel.v` solo contiene la infraestructura compartida (instancias de los tres módulos, clock, monitor de `o_lock`, tasks de reset/envío); cada uno de los 7 tests vive en su propio archivo bajo `tb/tests/`, nombrado según lo que verifica (por ejemplo `test_chained_transitions.v` para C5), envuelto como una task (`test_C0` .. `test_C6`) e incorporado vía ``include`. Se usó ``include` — y no una compilación separada por archivo — porque cada task necesita acceso directo a las señales del módulo “contenedor” (instancias de los DUT, registros de control). Cuál test correr se resuelve en tiempo de ejecución con un `case`, o con `+TEST=ALL`, corre la suite completa Co..C6 en orden — el comportamiento original de este testbench —, mientras que por ejemplo `+TEST=C3` corre únicamente ese test. Esto evita la recompilación por test que exige el patrón ``define TESTn / `include` usado en TP1 (sección 3): acá se compila una sola vez y se elige el test a correr (o todos).

Tests implementados:

Test	Archivo	Qué verifica
C0	test_direct_connection.v	Con <code>delay=0</code> : tras <code>i_soft_reset</code> del generador con seed <code>0xCAFE</code> , el checker logra lockear desde la nueva seed — instancia mínima que satisface la Actividad 5.
C1	test_fixed_distances.v	Lock a través de 5 distancias fijas de canal (0, 4, 8, 12, 16 ciclos).
C2	test_max_delay_traffic.v	Tráfico válido continuo con <code>delay=31</code> (latencia máxima soportada) durante un período completo (65535 datos válidos): el checker lockea y nunca se desbloquea.
C3	test_lock_unlock_boundaries.v	Fronteras de lock/unlock con <code>delay=7</code> : 4 datos válidos + 1 inválido → nunca lockea; ya lockeado, (2 inválidos + 1 válido) × 10 → nunca desbloquea.
C4	test_random_distance_transitions.v	8 iteraciones de lock→unlock, cada una con una distancia aleatoria distinta entre ráfagas, reseteando generador y checker antes de cada intento.
C5	test_chained_transitions.v	Transición encadenada: patrón (5 válidos + 3 inválidos) × 5, sin resetear generador/checker entre iteraciones, y con la distancia del canal re-randomizada antes de cada iteración — verifica que siempre alterna entre <code>LOCKED</code> y <code>UNLOCKED</code> bajo tráfico continuo y distancia variable, no solo tras un reset limpio a una latencia fija.
C6	test_reset_mid_traffic.v	Reset aleatorio del checker en medio de tráfico (no desde un estado limpio) × 5 — siempre vuelve a lockear.

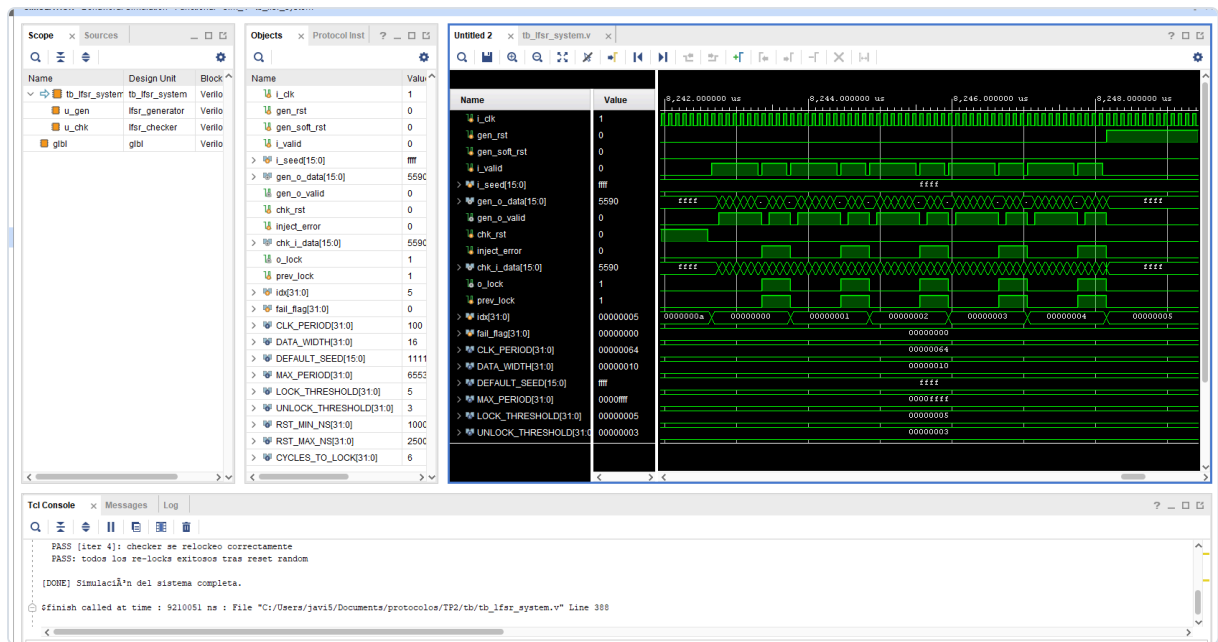


Figura 4. Consola de Vivado al finalizar la simulación del sistema, mostrando la última iteración (iter 4) del test de reset aleatorio del checker en medio de tráfico — el mismo patrón que hoy corre como TEST C6 en `tb_lfsr_system_channel.v` — con el mensaje final `PASS: todos los re-locks exitosos tras reset random` y el cierre de la simulación completa.

5. Extensiones

Estas modificaciones surgieron a partir de comentarios realizados en los encuentros del curso, orientados a lograr un código de mayor calidad, alineado con estándares de diseño y con mejores técnicas de verificación:

- `rtl/fsm_style/` : una reescritura del generador y el checker con el patrón de dos `always` separados (uno combinacional con `case` para el próximo estado, otro secuencial que solo registra), verificada como equivalente ciclo a ciclo (incluyendo estado interno completo) frente a las versiones originales, sin modificar ninguno de los testbenches existentes.
- **Latencia de canal configurable:** más allá del caso mínimo (`delay=0`), el modelo de canal soporta latencias de hasta 31 ciclos, ejercitadas en los tests C1 a C6 descritos en la sección anterior (distancias fijas, tráfico continuo, fronteras de lock/unlock y transiciones bajo latencia, y reset del checker en medio de tráfico con el canal activo).

Comparación de utilización de recursos — original vs. `fsm_style` : se sintetizó cada módulo por separado en sus dos versiones de escritura, para comparar la utilización de recursos resultante entre ambos estilos de RTL. Resultado: **utilización idéntica, celda por celda**, entre el original y `fsm_style` — las tablas siguientes no distinguen entre versiones porque no hay ninguna diferencia que mostrar.

Módulo	Slice LUTs	Slice Registers
<code>lfsr_generator</code>	11	17
<code>lfsr_checker</code>	33	25

Primitivas de `lfsr_generator` :

Primitiva	Cantidad	Categoría
IBUF	20	IO
OBUF	17	IO
FDPE	16	Flop & Latch
LUT3	13	LUT
LUT4	3	LUT
LUT2	2	LUT
FDCE	1	Flop & Latch
BUFG	1	Clock

Primitivas de `lfsr_checker` :

Primitiva	Cantidad	Categoría
FDCE	25	Flop & Latch
LUT5	22	LUT
IBUF	19	IO
LUT6	6	LUT
LUT4	6	LUT
LUT2	3	LUT
LUT3	2	LUT
CARRY4	2	CarryLogic
OBUF	1	IO
BUFG	1	Clock

Este resultado confirma que el sintetizador optimiza en base a la función lógica descrita (la tabla de verdad / la máquina de estados). Como ambas versiones ya estaban verificadas como equivalentes ciclo a ciclo —incluyendo el estado interno completo del checker—, describían exactamente la misma función, y Vivado llegó de forma independiente al mismo netlist óptimo para las dos. La reescritura en `fsm_style/` mejora la legibilidad y el apego a un patrón de codificación más estándar (combinacional separado de secuencial), pero no tiene costo ni beneficio en área para este caso en particular según como fueron escritos los módulos.

6. Resultados de verificación

Testbench	Tests	Resultado
tb_lfsr_generator.v	oa, ob, oc, 1, 2 (×5 seeds), 3, 4	15/15 PASS
tb_lfsr_system_channel.v	Co, C1 (×5 delays), C2, C3, C4 (×8 iter), C5, C6 (×5 iter)	20/20 PASS

Total: 35/35 PASS. Todas las simulaciones se verificaron mediante simulación funcional de los testbenches.

7. Conclusiones del TP2

El trabajo permitió aplicar en un caso concreto los conceptos de diseño de máquinas de estado (checker de 2 estados con umbral de lock/unlock), diseño de próximo estado combinacional vs. registro secuencial (tanto en el generador como en el checker), y sobre todo una metodología de verificación robusta: testbenches con tasks reutilizables, generación de estímulos aleatorios acotados (seeds, tiempos de reset, valid intermitente), medición de periodicidad como verificación matemática del diseño, e inyección deliberada de errores para probar los casos de frontera del checker.

A diferencia del TP1, en este TP2 tanto el diseño (generador, checker) como la verificación (testbenches y baterías de tests) se abordaron de manera más individual, guiada por el docente. A eso se sumaron las extensiones descritas en la sección 5, surgidas de comentarios realizados en los encuentros del curso.

Conclusiones generales

Ambos trabajos prácticos, aunque de alcance distinto, comparten una misma metodología de trabajo: **describir el RTL con separación clara entre lógica combinacional y secuencial, y verificar exhaustivamente ese RTL con testbenches propios.** El TP1 hace foco en el flujo de FPGA (síntesis, instrumentación de debug con VIO/ILA, generación de bitstream), mientras que el TP2 profundiza en el diseño puramente lógico y en la disciplina de verificación (cobertura de casos de borde, generación aleatoria de estímulos, y detección de bugs reales mediante simulación). En conjunto, cubren el ciclo de un flujo: especificación → RTL → verificación → síntesis/implementación en FPGA.